Experiment No. 4

Frequency Domain Image Filtering using Fourier Transform

Aim

To implement image filtering in the frequency domain using the Fourier Transform and analyze the effectiveness of frequency-based filtering techniques for noise removal, image enhancement, and feature preservation using Python and OpenCV.

Purpose

Many image processing operations can be performed more efficiently in the frequency domain than in the spatial domain. The Fourier Transform converts an image from the spatial domain into its frequency components, enabling selective manipulation of low-frequency and high-frequency information. Frequency domain filtering is widely used in medical image enhancement, satellite image analysis, image restoration, and biometric systems.

In this experiment, the Discrete Fourier Transform (DFT) is applied to an image, its frequency spectrum is analyzed, and Low-Pass and High-Pass Frequency Filters are implemented to observe their effects on image quality and feature extraction.

Prerequisites

- Basic knowledge of Digital Image Processing
- Understanding of spatial and frequency domains
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a grayscale image.
2. Compute the Discrete Fourier Transform (DFT) of the image using OpenCV or NumPy.
3. Shift the zero-frequency component to the center of the spectrum for visualization.
4. Display and analyze the magnitude spectrum.
5. Design and apply a Low-Pass Frequency Filter to suppress high frequencies and reduce noise.
6. Design and apply a High-Pass Frequency Filter to enhance edges and fine details.
7. Perform the Inverse Fourier Transform (IDFT) to reconstruct the filtered images.
8. Compare the original image, spectrum, low-pass filtered image, and high-pass filtered image.
9. Analyze the impact on smoothness, edge enhancement, and noise removal.
10. Document the observations and discuss the advantages of frequency domain filtering.

Key Formulas

- **2D DFT:** $F(u, v) = \text{sum over } x, y \text{ of } f(x, y)\, e^{-j2pi(ux/M + vy/N)}$
- **Inverse DFT:** $f(x, y) = (1 / MN) \text{ sum over } u, v \text{ of } F(u, v)\, e^{j2pi(ux/M + vy/N)}$
- **Magnitude spectrum (display):** $20 \log(|F(u, v)| + 1)$

- **Filtering:** $G(u, v) = H(u, v)\, F(u, v)$, then $g(x, y)$ = IDFT of G
- **Distance from centre:** $D(u, v) = \text{sqrt}((u - M/2)^2 + (v - N/2)^2)$
- **Ideal LPF:** $H = 1$ if $D <= D_0$, else 0
- **Gaussian LPF:** $H = \exp(-D^2 / 2D_0^2)$
- **Butterworth LPF:** $H = 1 / (1 + (D / D_0)^{2n})$
- **High-pass:** $H_{HP} = 1 - H_{LP}$

Observations

- The shifted magnitude spectrum has a bright center (low frequencies, overall brightness and smooth regions) and fainter outer areas (edges and fine detail). Strong horizontal and vertical lines in the spectrum correspond to edges perpendicular to them.
- Taking the DFT and then the inverse DFT without filtering recovers the original image (error is only rounding).
- Low-pass filtering blurs the image and reduces noise; a smaller cut-off D_0 gives stronger blur.
- The Ideal low-pass filter produces ringing artifacts around edges; Gaussian filtering gives none, and Butterworth is in between depending on its order n.
- High-pass filtering keeps edges and fine detail and removes smooth regions (the output is near zero mean); adding it back to the original sharpens the image.
- Choosing too small a D_0 removes real detail, while too large leaves noise; there is an optimal cut-off for noise removal.
- Periodic (stripe) noise appears as isolated spikes in the spectrum and can be removed almost completely with a notch filter, which is hard to do in the spatial domain.

Source Code

File: experiment4_frequency_filtering.py

```
"""
Experiment 4: Frequency Domain Image Filtering using Fourier Transform
Install:  pip install opencv-python numpy matplotlib
Usage:    python experiment4_frequency_filtering.py [path_to_image]
"""
import os
import sys
import cv2
import numpy as np
import matplotlib.pyplot as plt

os.makedirs("output", exist_ok=True)
rng = np.random.default_rng(0)

# ---------------------------------------------------------------
# Step 1: Load grayscale image
# ---------------------------------------------------------------
path = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

if img is None:  # fallback synthetic image
    print(f"'{path}' not found - generating a synthetic test image.")
    img = np.full((256, 256), 50, dtype=np.uint8)
    cv2.rectangle(img, (40, 40), (140, 170), 200, -1)
    cv2.circle(img, (190, 100), 40, 140, -1)
    cv2.putText(img, "FFT", (70, 230), cv2.FONT_HERSHEY_SIMPLEX, 1.5, 255, 3)

rows, cols = img.shape
print(f"Image shape: {img.shape}")


def psnr(ref, test):
    mse = np.mean((ref.astype(np.float64) - test.astype(np.float64)) ** 2)
    return float("inf") if mse == 0 else 10 * np.log10(255.0 ** 2 / mse)


# ---------------------------------------------------------------
# Steps 2-4: DFT, shift zero frequency to centre, magnitude spectrum
# ---------------------------------------------------------------
# (a) NumPy version
F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)                       # DC term now at the centre
magnitude = 20 * np.log(np.abs(F_shift) + 1)       # log scale for visualisation

# (b) OpenCV version (should give the same result)
dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft, axes=(0, 1))
mag_cv = 20 * np.log(cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1]) + 1)
print("NumPy vs OpenCV spectrum max difference:", np.abs(magnitude - mag_cv).max().round(4))

# ---------------------------------------------------------------
# Distance grid and filter masks H(u, v)
# ---------------------------------------------------------------
u = np.arange(rows) - rows // 2
v = np.arange(cols) - cols // 2
V, U = np.meshgrid(v, u)
D = np.sqrt(U ** 2 + V ** 2)                       # distance from centre


def ideal_lpf(D0):
    return (D <= D0).astype(np.float64)


def gaussian_lpf(D0):
    return np.exp(-(D ** 2) / (2 * D0 ** 2))


def butterworth_lpf(D0, n=2):
    return 1.0 / (1.0 + (D / D0) ** (2 * n))


# High-pass = 1 - low-pass
ideal_hpf = lambda D0: 1.0 - ideal_lpf(D0)
gaussian_hpf = lambda D0: 1.0 - gaussian_lpf(D0)
butterworth_hpf = lambda D0, n=2: 1.0 - butterworth_lpf(D0, n)


def apply_filter(image_fft_shifted, H):
    """Multiply spectrum by H, then inverse transform (IDFT) back to spatial domain."""
```

```
    G_shift = image_fft_shifted * H
    G = np.fft.ifftshift(G_shift)                # undo the centering shift
    g = np.real(np.fft.ifft2(G))                 # inverse DFT
    return g


def to_uint8(x):
    return np.clip(np.rint(x), 0, 255).astype(np.uint8)


# ---------------------------------------------------------------
# Steps 5-7: Low-pass and high-pass filtering + inverse DFT
# ---------------------------------------------------------------
D0 = 30
lp_ideal = to_uint8(apply_filter(F_shift, ideal_lpf(D0)))
lp_gauss = to_uint8(apply_filter(F_shift, gaussian_lpf(D0)))
lp_butter = to_uint8(apply_filter(F_shift, butterworth_lpf(D0)))

hp_ideal = apply_filter(F_shift, ideal_hpf(D0))
hp_gauss = apply_filter(F_shift, gaussian_hpf(D0))
hp_butter = apply_filter(F_shift, butterworth_hpf(D0))
# High-pass output has ~zero mean; rescale for display
hp_disp = lambda x: cv2.normalize(x, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Sharpening: add the high-pass result back to the original (high-boost)
sharpened = to_uint8(img + 1.0 * hp_gauss)

# Round-trip check: DFT -> IDFT without filtering must recover the image
recon = to_uint8(apply_filter(F_shift, np.ones_like(D)))
print("Reconstruction error (no filter):", np.abs(recon.astype(int) - img.astype(int)).max())

# ---------------------------------------------------------------
# Step 8-9: Noise removal with a low-pass filter, effect of cut-off D0
# ---------------------------------------------------------------
noisy = to_uint8(img + rng.normal(0, 25, img.shape))
Fn = np.fft.fftshift(np.fft.fft2(noisy))
print("\n---- PSNR vs clean image (dB) ----")
print(f"Noisy: {psnr(img, noisy):.2f}")
for d0 in (10, 20, 40, 80):
    out = to_uint8(apply_filter(Fn, gaussian_lpf(d0)))
    print(f"Gaussian LPF D0={d0:<3}: {psnr(img, out):.2f}")

# Periodic noise removal using a notch filter (a strength of the frequency domain)
x = np.arange(cols)
stripes = 40 * np.sin(2 * np.pi * 30 * x / cols)           # vertical stripes
periodic = to_uint8(img + np.tile(stripes, (rows, 1)))
Fp = np.fft.fftshift(np.fft.fft2(periodic))
mag_p = np.abs(Fp).copy()
mag_p[D < 10] = 0                                            # ignore DC / very low frequencies
pk = np.unravel_index(np.argmax(mag_p), mag_p.shape)         # strongest spike
notch = np.ones_like(D)
for (pu, pv) in [pk, (rows - pk[0], cols - pk[1])]:          # spike + its mirror
    Dk = np.sqrt((np.arange(rows)[:, None] - pu) ** 2 + (np.arange(cols)[None, :] - pv) ** 2)
    notch *= 1 - np.exp(-(Dk ** 2) / (2 * 4 ** 2))
notch_out = to_uint8(apply_filter(Fp, notch))
print(f"\nPeriodic noise PSNR: {psnr(img, periodic):.2f} -> notch filtered: {psnr(img, notch_out):.2f}")

# ---------------------------------------------------------------
# Display
# ---------------------------------------------------------------
fig, ax = plt.subplots(2, 4, figsize=(15, 7.5))
items = [
    ("Original", img), ("Magnitude spectrum (log, shifted)", magnitude),
    (f"Ideal LPF D0={D0}", lp_ideal), (f"Gaussian LPF D0={D0}", lp_gauss),
    (f"Butterworth LPF D0={D0}", lp_butter), (f"Ideal HPF D0={D0}", hp_disp(hp_ideal)),
    (f"Gaussian HPF D0={D0}", hp_disp(hp_gauss)), ("Sharpened (img + HPF)", sharpened),
]
for a, (t, im) in zip(ax.ravel(), items):
    a.imshow(im, cmap="gray"); a.set_title(t, fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("output/exp4_filters.png", dpi=150); plt.show()

# Filter masks and their effect on the spectrum
fig, ax = plt.subplots(1, 4, figsize=(15, 4))
for a, (t, H) in zip(ax, [("Ideal LPF mask", ideal_lpf(D0)), ("Gaussian LPF mask", gaussian_lpf(D0)),
                          ("Butterworth LPF mask", butterworth_lpf(D0)),
                          ("Spectrum x Gaussian LPF", 20 * np.log(np.abs(F_shift * gaussian_lpf(D0)) + 1))]):
    a.imshow(H, cmap="gray"); a.set_title(t, fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("output/exp4_masks.png", dpi=150); plt.show()

# Ringing artefact of the ideal filter vs smooth Gaussian
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for a, (t, im) in zip(ax, [("Original", img), ("Ideal LPF D0=15 (ringing)", to_uint8(apply_filter(F_shift, ideal_lpf(15)))),
                           ("Gaussian LPF D0=15 (no ringing)", to_uint8(apply_filter(F_shift, gaussian_lpf(15))))]):
    a.imshow(im, cmap="gray"); a.set_title(t, fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("output/exp4_ringing.png", dpi=150); plt.show()
```

```
# Noise removal and periodic-noise (notch) results
fig, ax = plt.subplots(1, 4, figsize=(15, 4))
for a, (t, im) in zip(ax, [("Noisy (Gaussian)", noisy), ("Gaussian LPF D0=40", to_uint8(apply_filter(Fn, gaussian_lpf(40)))),
                           ("Periodic stripe noise", periodic), ("Notch filtered", notch_out)]):
    a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t, fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("output/exp4_noise.png", dpi=150); plt.show()
```

Questions and Answers

1. What is the Fourier Transform? Why is it important in digital image processing?

The Fourier Transform decomposes a signal into a sum of sinusoids of different frequencies, amplitudes, and phases. For an image, it converts the pixel data into a representation of spatial frequencies: low frequencies are smooth, gradual changes and high frequencies are edges, textures, and noise. It is important because it:

- Allows selective removal or enhancement of frequency components (smoothing, sharpening, denoising)
- Makes large-kernel convolution efficient, since convolution becomes multiplication
- Helps analyze texture, periodic patterns, and image structure
- Is the basis for restoration, compression, and image registration

2. Differentiate between the spatial domain and the frequency domain.

Aspect	Spatial Domain	Frequency Domain
Representation	Pixel intensities f(x, y) at positions in the image	Spatial frequency components F(u, v) with magnitude and phase
Operations	Applied directly to pixels or neighborhoods (convolution with kernels)	Applied to the transform coefficients (multiplication by H(u, v))
Information	Where things are	How fast the intensity changes (what frequencies are present)
Good for	Local operations, point operations, median filtering	Periodic noise, global filtering, large kernels, restoration

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Digital images are discrete and finite, so the DFT is the form of the Fourier Transform that can be computed on them. It gives a finite set of frequency coefficients that can be modified and inverted exactly. With the Fast Fourier Transform (FFT) algorithm, the cost drops from $O(N^4)$ to $O(N^2 \log N)$ for an N x N image, making frequency-domain filtering practical. It enables filtering, spectrum analysis, convolution via multiplication, and applications such as compression and restoration.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

In the raw DFT output, the zero-frequency (DC) term is at the top-left corner and the low frequencies are split across the four corners. Shifting (`fftshift`) moves the DC term to the center so that frequency increases radially outward. This gives a more intuitive view of the spectrum, and makes it easy to design circularly symmetric filters (distance from the center D(u, v)) and to locate periodic-noise spikes for notch filters. Before the inverse transform, the shift must be undone with `ifftshift`.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Aspect	Low-Pass Filter	High-Pass Filter
Passes	Low frequencies (below cut-off D0)	High frequencies (above cut-off D0)
Effect	Smoothing, blurring, noise reduction	Edge and detail enhancement, sharpening
Types	Ideal, Butterworth, Gaussian LPF	Ideal, Butterworth, Gaussian HPF (1 - LPF)
Applications	Denoising, image smoothing, preprocessing before downsampling, removing high-frequency scan noise	Edge detection, sharpening of medical and satellite images, feature extraction, fingerprint ridge enhancement

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

The IDFT converts the (modified) frequency-domain data back to the spatial domain so that the filtered result can be viewed and used. Since the DFT/IDFT pair is lossless, the IDFT reproduces the original image exactly when no filtering is applied, and after filtering it yields the image with the chosen frequency components kept or removed. Both magnitude and phase are needed for correct reconstruction, and the real part is taken because the imaginary part is only numerical error.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

- Periodic noise (stripes, moire patterns) appears as isolated spikes and can be removed precisely with notch filters
- It gives exact control of which frequency bands are kept, with filters of desired shape (Gaussian, Butterworth)
- For large kernels, multiplication in the frequency domain is faster than spatial convolution
- It is a natural framework for restoration (inverse and Wiener filtering) and for analyzing blur and noise
- It offers a clear understanding of what the filter does to each frequency

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

1. Medical imaging: MRI and CT reconstruction and enhancement
2. Image compression: JPEG (DCT, a Fourier-related transform)
3. Satellite and remote sensing: removing striping and periodic noise, texture analysis
4. Biometrics: fingerprint and iris recognition (ridge frequency analysis)
5. (Also: image registration by phase correlation, deblurring, and pattern or texture analysis)

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Aspect	Spatial Domain	Frequency Domain
Computation	Convolution costs $O(N^2k^2)$; efficient for small kernels (3x3, 5x5)	FFT, multiply, inverse FFT costs $O(N^2 \log N)$, independent of kernel size; efficient for large kernels
Implementation	Simple and intuitive; supports non-linear filters (median)	Needs transform, padding, and handling of complex data; linear filters only
Real-time use	Preferred in real-time and embedded systems	Preferred for offline or large-image processing
Applications	Smoothing, edge detection, impulse noise removal	Periodic noise removal, restoration, deblurring, band-selective filtering, compression

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

- **Restoration:** degradation (blur, periodic noise) is modeled as multiplication in the frequency domain, so inverse and Wiener filters can undo it directly, and notch filters remove periodic interference without harming other content.
- **Feature extraction:** high-pass and band-pass filters isolate edges, ridges, and textures at chosen scales (e.g., Gabor filters, fingerprint ridges), while low-pass filtering removes noise that would give false features.
- Controlled cut-offs and smooth filters (Gaussian, Butterworth) avoid ringing and preserve useful structure, so downstream detectors and classifiers get cleaner, more consistent inputs.

Conclusion

The DFT of an image was computed and visualized, and Ideal, Gaussian, and Butterworth low-pass and high-pass filters were implemented and applied using the inverse DFT. Low-pass filters remove noise at the cost of blur, high-pass filters enhance edges and details, the Ideal filter causes ringing while Gaussian does not, and a notch filter removes periodic noise very effectively. Frequency domain filtering is therefore especially useful for restoration and for noise that has a clear frequency signature.